

LEHRSTUHL FÜR RECHNERARCHITEKTUR UND PARALLELE SYSTEME

Grundlagenpraktikum: Rechnerarchitektur

Gruppe 128 – Abgabe zu Aufgabe A215

Sommersemester 2023

Jonathan Schwarzer

Johannes Linke

Timothy Davies

1 Einleitung

Eine raumfüllende Kurve ("spacefilling curve") bezeichnet eine Linie, welche eine zweidimensionale Fläche oder einen mehrdimensionalen Raum komplett durchläuft. Die erste Abbildung jener Kurve wurde durch Giuseppe Peano im Jahr 1890 gezeigt. Die erste graphische Darstellung einer solchen Kurve wurde durch David Hilbert ein Jahr später dargestellt [3]. Im Laufe der Zeit wurden weitere raumfüllende Kurven entwickelt. Dabei kann das Prinzip der raumfüllenden Kurve neben einem zweidimensionalen Raum, auch auf mehrdimensionale Räume angewandt werden. Das Ziel der Peano-Kurve ist es, eine kontinuierliche Abbildung vom Einheitsintervall $([0,1])$ auf das Einheitsquadrat zu konstruieren [3]. Dabei entsteht eine Linearisierung n -dimensionaler Daten [4]. Dadurch wird eine Abbildung von Punkten eines n -dimensionalen Raumes auf ein Intervall der reellen Zahlen erzeugt. Durch diese Eigenschaft entsteht eine „Lokalitätserhaltung“ welche in mehreren Anwendungsgebieten genutzt wird. Ein großes Anwendungsgebiet ist dabei die Bildbearbeitung. Dabei werden mittels der raumfüllenden Kurven Operationen auf den einzelnen Pixel eines Bildes ausgeführt. Eine Arbeit von Zang et al. stellen diesen Prozess dar. Mittels der Hilbert-Kurve werden dort Glättungsoperationen auf Pixel angewandt. Sie konnten dabei feststellen, dass dieses Problem durch die raumfüllende Kurve effizient gelöst werden kann. Da anstelle eines zweidimensionalen Bildrasters auf einer eindimensionalen Kurve gearbeitet wird [5]. Während dieses Problem mittels der Hilbert-Kurve gelöst wurde, kann hierbei auch die Peano-Meander Kurve angewandt werden. Auf diese wird in dieser Arbeit näher eingegangen. Ziel dieser Arbeit ist es einen Algorithmus zu entwickeln, welcher für eine Eingabe n eine geordnete Liste an ganzzahligen Koordinaten ausgibt, welche die Abfolge der Knotenpunkte der Peano-Meander-Kurve n -ten Grades repräsentiert. Im Verlauf der Arbeit wurden mehrere Implementationsmöglichkeiten festgestellt und analysiert. Dabei basieren die Implementierungen auf verschiedenen Vorgehensweisen. Neben einer naiven rekursiven Implementierung, wird eine optimierte rekursive Implementierung mittels SIMD-Vektorisierung dargestellt. Abschließend wird eine iterative Lösungsmöglichkeit implementiert. Es konnte festgestellt werden, dass die iterative Implementierung deutlich effizienter die Berechnungen durchführt. Ebenfalls wird deutlich, dass die naive rekursive Implementierung am längsten zur Erstellung der Kurve benötigt.

2 Lösungsansatz

Im folgenden Teil wird zunächst auf den mathematischen Hintergrund einer raumfüllenden Kurve eingegangen. Des Weiteren wird die naive rekursive Implementierung, die rekursiv optimierte Implementierung und die iterative Implementierung betrachtet.

2.1 Mathematischer Hintergrund

Der Grenzwert einer Folge von Kurven, die schrittweise konstruiert werden können, wird als die Peano-Kurve bezeichnet. Die sogenannte raumfüllende Kurve (FASS-Kurve), passiert dabei jede Ecke eines Einheitsquadrats [1]. Im zweidimensionalen Fall wird ein Quadrat in neun gleich große Quadrate unterteilt. Jedes der Quadrate wird durchlaufen (vgl. Abbildung 1). Im nächsten Schritt werden die Quadrate auf neun weitere Quadrate unterteilt und wieder durchlaufen (vgl. 2). Dabei beschreibt n den Grad der Kurve. Während für $n=1$ 9 Quadrate genutzt werden, sind es für $n=2$ 81 einzelne Quadrate (vgl. Abbildung 2). Somit kann die Anzahl der Quadrate durch folgende Gleichung festgestellt werden:

$$\text{Anzahl Quadrate} = 9^n \quad (1)$$

Durch die Darstellung im zweidimensionalen Raum müssen somit 9^n Punkte bestimmt werden. Durch Drehungen und Spiegelungen der Peano-Meander-Kurve mit Grad Eins (vgl. Abbildung 1) werden Peano Kurven mit $n > 1$ erzeugt. Dabei erhält man Grenzwerte, welche auf dem Ausgangsquadrat liegen und unendlich lang sind. Des Weiteren liefert es eine surjektive stetige Abbildung $f = I \rightarrow I \times I$.

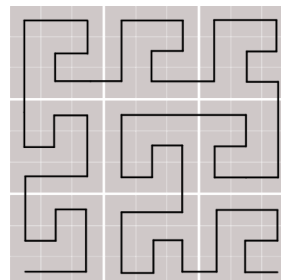
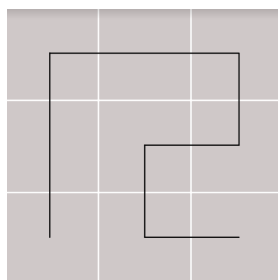


Abbildung 1: Peano-Meander-Kurve $n = 1$ **Abbildung 2:** Peano-Meander-Kurve $n = 2$

2.2 Rekursiver Ansatz

Der rekursive Ansatz bildet die Grundlage für weitere Vorgehensweisen zur Bildung der Peano-Meander-Kurve. Dabei basiert der Ansatz auf der Implementierung der Hilbert-Kurve. Die Peano-Meander Kurve basiert auf einem $3n \times 3n$ Array und durchläuft niemals mehr als fünf aufeinanderfolgende Punkte in dieselbe Richtung [2]. Die Basis der Implementiert basiert auf der Verteilung der Kurve auf einzelne Teilbereiche. Dabei die Position und Rotierung jeder einzelnen Kurve durch ihre Position in der Kurven-erzeugung definiert. Um die korrekte Drehung jener Kurve zu bestimmen wurden die

Variablen `startPoint` und `endPoint` genutzt. Anhand der Abbildung 3 werden alle vier Drehmöglichkeiten mit Start- und Endpunkten dargestellt. Die Variable `startPoint` bekommt dabei den Wert Null wenn der Startpunkt in der linken unteren Ecke sein soll. Für die obere rechte Ecke wird der `startPoint` auf den Wert Eins gesetzt. Die Variable `endPoint` wird demnach auf Null gesetzt, wenn der Endpunkt unten Rechts oder unten Links liegen soll. Bei einem Endpunkt auf der linken oder rechten oberen Ecke wird `endPoint` mit Eins deklariert. Auf Basis der beiden Punkte kann man im nächsten Schritt die Form der Peano-Meander-Kurve definieren. Dafür werden mittels Rekursion die Methoden aufgerufen und bei jedem Aufruf die Parameter je nach Kurvenabschnitt verändert. Mit dem Aufruf der Methode `peano_meander` werden durch Operationen im Methodenkopf die X und Y Koordinate berechnet. Da für jeden Aufruf ein Strich in der Kurve gebildet wird, müssen neun Methodenaufrufe stattfinden um eine komplette Kurve zu bilden. Sobald die kleinste Form des Quadrats erreicht wird, bricht der rekursive Aufruf ab und die Koordinaten werden in einem Array gespeichert. Diese Art der Implementierung bildet die Grundlage für weitere Vorgehensweisen. Zum einen wird im folgenden Teil auf Optimierungsmöglichkeiten basierend auf dem rekursivem Ansatz eingegangen.

```
1 void peano_Notoptimized(coord_X+(2*startPoint*width), coord_Y+(2*
    startPoint*width), width, startPoint, endPoint, x, y);
```

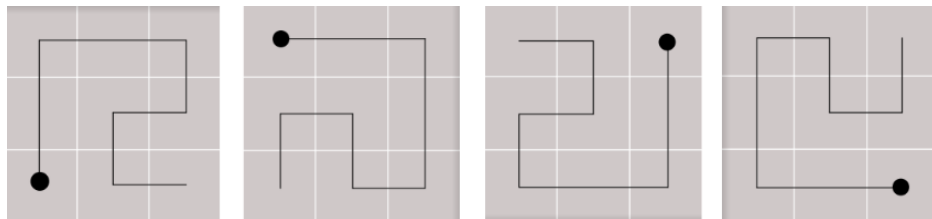


Abbildung 3: Vier möglichen Drehungen der Peano-Meander Kurve

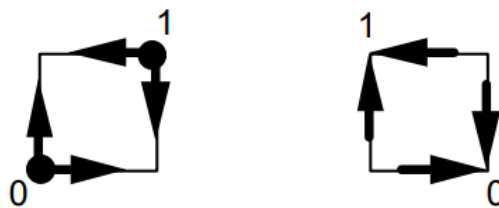


Abbildung 4: Start -und Endpunkt Quelle: Breinholt und Schierz, p. 2

2.3 Rekursiv optimierter Ansatz

Der Grundgedanke bei der optimierten rekursiven Variante besteht darin, bereits berechnete Teilprobleme zu speichern und sie bei Bedarf wiederzuverwenden. Dies ermöglicht eine schnellere Berechnung der nächsten Koordinaten mithilfe von SIMD (Single Instruction, Multiple Data). Im folgenden Abschnitt wird der Entstehungsprozess dieser Optimierung beschrieben.

Beim Betrachten der Peano-Meander-Kurve fällt auf, dass sich das zugrundeliegende Muster immer wiederholt, jedoch in unterschiedlichen Formen. Insgesamt treten vier

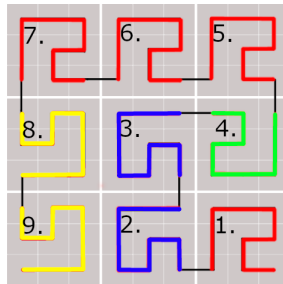


Abbildung 5: Teilfiguren eingezeichnet

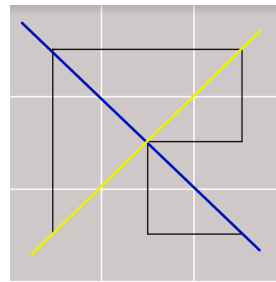


Abbildung 6: Spiegelachsen

unterschiedliche Transformationen auf, wie in Abbildung 5 farblich gekennzeichnet. In Rot sind die Teilbereiche gekennzeichnet, bei denen keine weitere Veränderung nötig ist. Bei Blau muss an der blauen Spiegelachse (vgl. Abbildung 6) gespiegelt werden. Analog bei der gelben Figur an der gelben Spiegelachse (vgl. Abbildung 6). Um die letzte Teilfigur durch grundlegende Transformationen zu erhalten, muss die Grundfigur um 180° gedreht werden. Auf den beiden Abbildung wird der Prozess der berechnung von $n = 2$ beschrieben. Dieses Vorgehen lässt sich auch für größere n identisch anwenden. Man kann hier nun ausnutzen, dass nicht jede Koordinate einzeln berechnet werden muss, sondern diese Transformationen auch mittels SIMD auf mehreren Koordinaten gleichzeitig anwendbar sind.

Im Programm wird also letztendlich rekursiv die erste Figur mit $n = 1$ berechnet. Da die Länge der Teilfigur bekannt ist, kann man auf die Positionen der nächsten 8 Teilfiguren im Array schließen. Somit kann man aus der ersten Teilfigur die x- und y-Koordinaten kopieren, sie dann passend transformieren und eventuell gleich an mehreren Stellen in das Array schreiben.

```
1 void transform_v1(coord_t *x_values, coord_t *y_values, size_t length) {
2     copy_coords(x_values, y_values, length * 4, 0, length);
3     copy_coords(x_values, y_values, length * 5, 0, length);
4     copy_coords(x_values, y_values, length * 6, 0, length);
5 }
```

Die Methode `transform_v1` beispielsweise kopiert die erste Teilfigur der Länge `length` an die 5., 6. und 7. Position. Die Position der Teilfigur (vgl. Abbildung 3) muss um 1 verringert werden, da, wie bei Arrays üblich, bei 0 angefangen wird zu zählen. Die

anderen transform-Methoden beruhen auf dem gleichen Prinzip und spiegeln bzw. rotieren die Teilfigur entsprechend.

Letztlich müssen jedoch die 8 Teilfiguren an die richtige Stelle verschoben werden. Dies geschieht dann in den rekursiven Aufrufen der Methode. Die x- und y-Koordinaten werden dabei übergeben. Die Teilfiguren müssen dann nur noch um die Koordinaten verschoben werden, da sie bereits richtig transformiert im Array stehen. Zusammenfassend kann man sagen, man nimmt immer eine Basisfigur, diese wurde zuvor rekursiv berechnet und diese Basisfigur wird nun 8 mal kopiert und auf verschiedene Weisen transformiert und verschoben. Die nun berechnete Figur hat neun mal so viele Koordinaten und bildet die nächste Basisfigur.

2.4 Iterativer Ansatz

Die dritte in diesem Projekt erarbeitete Implementierung, basiert auf der schrittweisen Konstruktion der Kurve durch Schließen und iterativen Berechnungen anstatt der zuvor verwendeten Rekursion. Der Grundgedanke dieses Ansatzes basiert ähnelt sich der in Abschnitt 2.3 vorgestellten Variante. Auch hier wird der nächstgrößere Kurvengrad durch Transformationen der vorhergehenden Kurve berechnet. Anstatt wie bei der Rekursion, sich von dem gewünschten Grad der Kurve nach unten bis $n = 1$ zu arbeiten, wird in dieser Variante bei $n = 1$ angefangen und nach oben gearbeitet. Dadurch werden pro Iterationsschritt nur noch 8 Teilfiguren benötigt, welche eingefügt werden müssen. Aufgrund der Funktionsweise ist bei jeder Iteration die Distanz, um welche die 8 Teilfiguren verschoben werden, bekannt. Das ermöglicht, während dem Transformieren die Koordinaten der Teilfiguren zeitgleich zu verschieben. Dieses Vorgehen reduziert somit die Anzahl der Speicherzugriffe und die Anzahl Iterationen über die Koordinaten der Teilfiguren. Somit kann schneller die Peano-Meander Kurve nächst höheren Grades berechnet werden. Wobei diese nun die Basis für die darauffolgende Iteration bildet.

3 Genauigkeit

Die Resultate der Kurve sind eindeutig, da jegliche Algorithmen welche diese generieren, das gleiche Resultat liefern müssen, um als Peano-Meander-Kurve klassifiziert zu werden. Die Generierung der Kurve jedoch, ist nicht eindeutig, da ein Algorithmus zum Berechnen von links als auch von rechts unten, den Start- und Endpunkten, beginnen können ohne die Resultate zu verändern.

Im Verlauf der Implementierung der Peano-Meander Kurve kam es zu Fehlern für größere n der Kurve. Bei der Analyse des Codes wurde festgestellt, dass die Verwendung der Funktion `pow(double, double)` aus der C Bibliothek `math.h` für große Inputparameter ungenaue Werte lieferte. Die `pow`-Funktion ist nicht für den Integer-Datentyp definiert. Da in der Implementierung nur Ganzzahlen genutzt werden und diese von `pow()` nicht zurückgegeben werden, musste ein alternativer Ansatz zur Potenzierung genutzt werden. Die Funktion `fast_power(int, int)` löst dieses Problem. Dabei benötigt die Funktion

zwei Ganzzahlen und gibt als Rückgabe einen "long long" Wert zurück. Demnach wird sichergestellt, dass bis zu 64-bit Zahlen abgedeckt werden. Durch das Vermeiden von Fließkommazahlen, kommt es zu keinen Rundungsfehler.

4 Performanzanalyse

4.1 Testumgebung

Im folgenden Teil wird zunächst auf die Testumgebung eingegangen. Des Weiteren werden Limitationen in Bezug auf die Ausführung der Performencetests dargestellt. Abschließend werden die Performanceergebnisse der einzelnen Implementierung dargestellt und miteinander verglichen. Wichtige Komponenten in der Performance Optimierung und Ausführung ist der verwendete Arbeitsspeicher und Prozessor, insbesondere bei der Ausführung von rechenintensiven Programmen. Daher wird in diesem Zusammenhang 16-Gigabyte-DDR4-3200-MHz-RAM in Dualchannel-Konfiguration (2x 8GB) in Verbindung mit Ryzen 7 3800X 8 - Core, 16 Threads Prozessor benutzt. Alle Tests werden im WSL 2-Subsystem von Windows 11 ausgeführt. Der zur Verfügung gestellte Compiler, GCC (GNU Compiler Collection), hat unter zuhilfenahme des Flags -O3 den Code optimiert und vektorisiert. Somit werden Teile des Programms mittels SIMD ausgeführt, was die Performance zusätzlich verbessern kann. Alle Eingaben wurden 20 mal wiederholt, wobei die durchschnittliche Laufzeit in den folgenden Diagrammen verwendet wird.

4.1.1 Einschränkungen

Aufgrund der beschränkung des RAMs auf 16GB war das Testen nur bis zum Kurvengrad $n = 9$ möglich. Problematisch ist, dass bei Grad $n = 10$ bereits 27GB RAM nötig wären. Die 27GB für Speicher setzen sich wie folgt zusammen:

$$A_{\text{Anzahl Quadrate}} = 9^n \quad (2)$$

$$A_{\text{Anzahl Integer}} = A_{\text{Anzahl Quadrate}} \times 2 \quad (3)$$

Angenommen ein Integer ist 4 byte groß:

$$A_{\text{Anzahl Bytes}} = A_{\text{Anzahl Integer}} \times 4 \quad (4)$$

Für $n = 10$ wären es in etwa 27,9GB RAM. Selbst mit einem 32GB RAM System wäre es nicht möglich das zu Testen, aufgrund des Overheads durch das Betriebssystem.

4.2 Rekursiv

Mittels Abbildung 8 wird die jeweilige Dauer der Ausführung für $n = 1$ bis $n = 9$ dargestellt. Dabei beschreibt die y-Achse die Dauer in Millisekunden und die x-Achse die Größe der Kurve. Aus der Abbildung wird ersichtlich, dass die Dauer der Ausführung mit zunehmender Größe der Kurve ansteigt. Besonders interessant ist, dass sich die

Ausführungsdauer zwischen den Kurven um das Zehnfache erhöht. Dieser Anstieg kann auf die Vorgehensweise der Rekursion zurückgeführt werden, die für jede zusätzliche Kurvengröße notwendig ist. Bei allen ausgeführten Tests hatte das Programm einen exponentiellen Anstieg der Ausführungszeit. Während die Gesamtzeit bei $n = 8$ etwa 13 Sekunden beträgt, beendet das Programm nach etwa 27 Sekunden bei $n = 9$, mit einer Durchschnittszeit von 1381 (1.3 Sekunden) Millisekunden pro Ausführung.

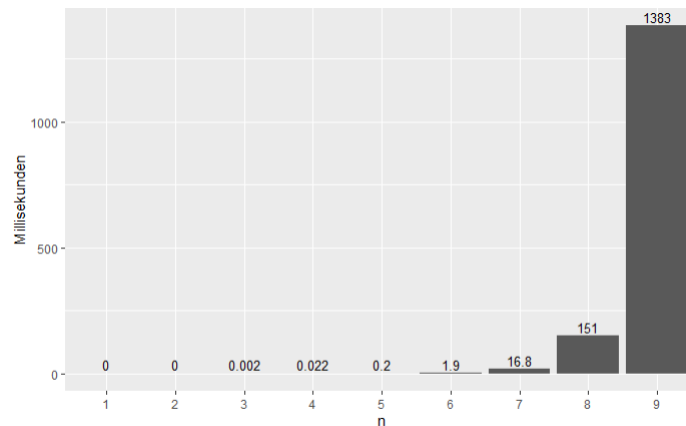


Abbildung 7: Durchschnittlich gemessene Zeit für $n = 1$ bis $n = 9$

4.3 Rekursiv optimierter Ansatz

Bei dem Rekursiv optimierter Ansatz ist ein starker exponentieller Anstieg an Laufzeit mit steigendem n zu erkennen. Von $n = 1$ bis $n = 6$ ist die Laufzeit unter 1 ms. Die Differenz der Laufzeit von $n = 1$ zu $n = 9$ ist beträchtlich. Bei $n = 1$ ist die Laufzeit nicht mehr Messbar, wobei man bei $n = 10$ fast 500 ms hat. Auffällig ist auch, dass sich die Laufzeit grob verzehnfacht, wobei in dem Schritt von $n = 6$ zu $n = 7$ sie sich verzwanzigfacht.

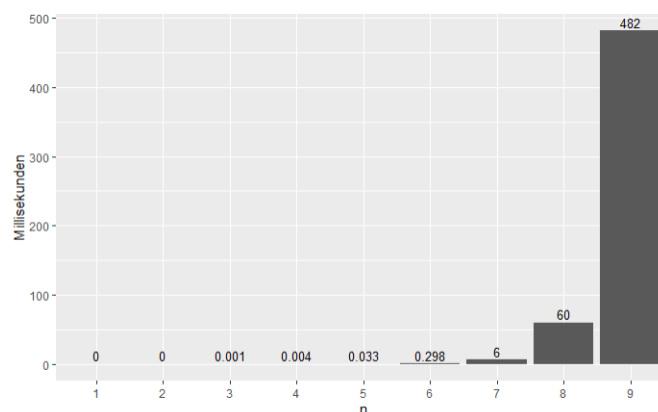


Abbildung 8: Durchschnittlich gemessene Zeit für $n = 1$ bis $n = 9$

4.4 Iterativer Ansatz

Der Iterative Ansatz hat, wie in Abbildung 9 zu sehen ist, bei Grad = 9 eine Durchschnittliche Ausführungszeit von 278ms und ist somit die schnellste der drei vorgestellten Implementierungen.

Der Code kann aufgrund seiner iterativen Art und der Verwendung von for-Schleifen die Vorteile von SIMD-Instruktionen vollständig ausnutzen. Dadurch wird selbst bei kleinen n schon viel Laufzeit eingespart, da schon bei $n = 3$ insgesamt 729 verschiedene X-Koordinaten und gleich viele Y-Koordinaten berechnet werden müssten.

Dazu kommt noch, dass der Overhead für das erstellen und Abarbeiten der Stack-Frames, wie es bei Rekursiven Varianten vorkommen kann bei einer rein iterativen Implementierung wegfällt. Funktionsaufrufe wurden zudem auf das absolute minimum reduziert um auch hier den Overhead von Call-Befehlen zu vermeiden. Was bei den Durchschnittlichen Ausführungszeiten von $n = 1$ bis $n = 9$ das Wachstum immer zwischen einem Faktor von 9 und 10 liegt. Dieses lässt sich zurückführen auf das 9-fache kopieren und bearbeiten der vorherigen Iteration, um die nächste zu berechnen. Das der Wachstumsfaktor größer als 9 ist, lässt sich begründen mit dem wiederholten ausführen und aufrufen der Transformationsmethoden pro Iteration und dem neu berechnen der `move_by` und `one_ninth` Variablen.

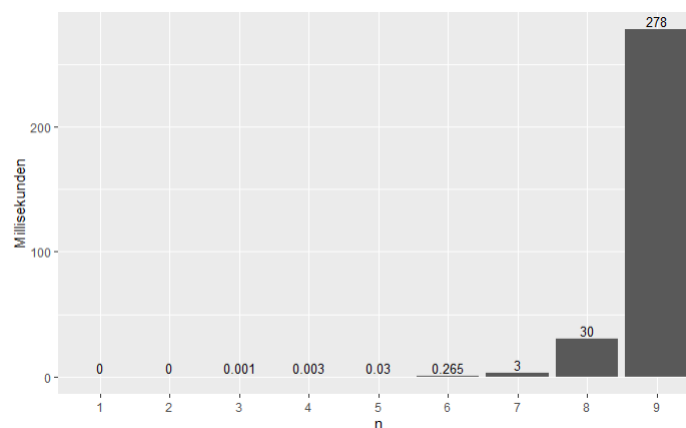


Abbildung 9: Durchschnittlich gemessene Zeit für $n = 1$ bis $n = 9$

4.5 Interpretation

Abbildung 10 stellt die entsprechenden Laufzeiten der einzelnen Implementierungen dar. Die rekursive Variante zeigt eine exponentielle Steigerung der Laufzeit mit jedem weiteren Grad der Kurve. Dies ist darauf zurückzuführen, dass die Anzahl der zu berechnenden Punkte bei jedem weiteren Grad verneunfacht. Bei dieser Implementation muss jedoch jede Koordinate einzeln berechnet werden, was die SIMD-Vektorisierung ausschließt.

Beim rekursiv optimierten Ansatz ist eine verzehnfachung der Laufzeit festzustellen. Im

Vergleich zur ersten rekursiven Implementierung ermöglicht dieser Ansatz jedoch die Nutzung von SIMD-Instruktionen. Das Kopieren und Transformieren der Koordinaten der Basisfigur im Array kann auf mehreren Koordinaten gleichzeitig durchgeführt werden. Allerdings verlangsamt die rekursive Struktur des Algorithmus die Laufzeit, da die richtigen Positionen der acht Teilfiguren zunächst bestimmt werden müssen. Jede Koordinate jeder Teilfigur muss somit individuell um den richtigen Wert verschoben werden. Dies kann zwar unter Verwendung von SIMD-Befehlen erfolgen, erfordert jedoch zusätzliche Speicherzugriffe.

Neben den rekursiven Implementierungen benötigt der iterative Ansatz eine deutlich geringere Laufzeit. Wie bei den vorangegangenen Implementierungen wird auch hier die Laufzeit mit steigendem n verzehnfacht. Im Vergleich zum rekursiv optimierten Ansatz ist die iterative Variante etwa 50 Prozent schneller und im Vergleich zum naiven rekursiven Ansatz etwa 400 Prozent schneller. Der Performance-Unterschied resultiert sowohl aus dem fehlenden Overhead durch rekursive Aufrufe, als auch die korrekte Transformation und Verschiebung der Teilfiguren an die richtigen Stellen. Dadurch können zusätzliche Speicherzugriffe vermieden werden, die sich negativ auf die Performance auswirken würden. Aufgrund der SIMD-Vektorisierung, der reduzierten Speicherzugriffe und des fehlenden Overheads durch rekursive Aufrufe ist die iterative Variante die schnellste der Peano-Meander Implementierungen.

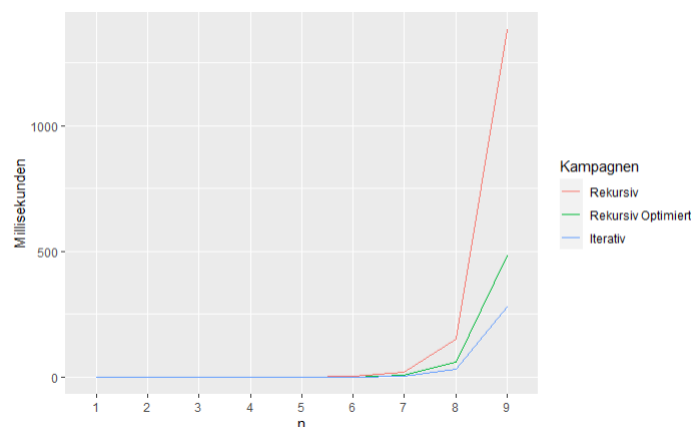


Abbildung 10: Laufzeit aller Implementierungen

5 Zusammenfassung und Ausblick

Diese Arbeit befasste sich mit der Konstruktion der Peano-Meander-Kurve. Diese stellt eine raumfüllende Kurve da, welche eine zwei-dimensionale Fläche komplett durchläuft. Das Ziel der Peano-Kurve ist es, eine kontinuierliche Abbildung von einem Einheitsintervall auf ein Einheitsquadrat zu erzeugen. Sie findet dabei oft Anwendung im Bereich der Bildbearbeitung. Neben zweier rekursiven Implementierungen wurde ebenfalls auf einen iterativen Ansatz zurückgegriffen. Mittels der Performanceanalyse wurde

festgestellt, dass alle Implementierungen eine exponentielle Steigung in Bezug auf die Laufzeit haben. Jedoch unterscheiden sich alle drei in Form der Länge der Durchführung. Während die naive rekursive Implementierung die längste Laufzeit aufweist, zeigt die optimierte rekursive Implementierung eine deutlich schnellere Laufzeit. Durch SIMD-Vektorisierung kann ein Großteil der Berechnung vermieden werden, da auf schon berechnete Ergebnisse zurückgegriffen wird. Jedoch wird die Laufzeit, wegen dem Nutzen von Rekursion im ersten Schritt der Berechnung, verlangsamt. Der iterative Ansatz erzielt die beste Performance. Durch die Vermeidung von rekursiven Aufrufen und die korrekte Transformierung und Verschiebung der Teilfiguren werden zusätzliche Speicherzugriffe vermieden. Dabei nutzt der iterative Ansatz ebenfalls die Vorteile von SIMD-Vektorisierung.

Die vorgestellten Implementierungen der Peano-Meander Kurve liefern bereits gute Ergebnisse hinsichtlich der Effizienz und Performance. Dennoch gibt es hierbei noch Raum für weitere Verbesserungen und Erweiterungen. Demnach könne die Skalierbarkeit der Implementierung weiter untersucht werden. Derzeit sind alle Implementierungen auf einen Arbeitsspeicher von 16GB beschränkt und werden für $n > 9$ nicht ausgeführt. Es wäre demnach interessant zu untersuchen, welchen Einfluss ein größerer Arbeitsspeicher oder eine Methode für bessere Speicherverwaltung, auf die Laufzeit hat. Ebenfalls wäre es interessant zu wissen, in welchen weiteren Gebieten der Einsatz der Peano-Meander-Kurve von Vorteil sein kann. Insgesamt bietet die Peano-Meander-Kurve ein spannendes Forschungsgebiet mit vielfältigen Möglichkeiten. Die untersuchte Implementierung bilden dabei den Ausgangspunkt für weitere Entwicklungen.

Literatur

- [1] Bianca Hoegel. Peano-kurve. 2021. <https://www.biancahoegel.de/geometrie/fraktal/peano-kurve.html>, Zugriff 02.07.2023.
 - [2] Greg Breinholt and Christoph Schierz. A recursive algorithm for the generation of space-filling curves. January 1998.
 - [3] Jens-Michael Wierum. Anwendung diskreter raumfullender kurven. Oktober 2003.
 - [4] Peter Oberhofer. Anwendung raumfullender kurven und parallelisierung. Juni 2003. p. 3.
 - [5] Yu Zang, Hua Huang, Lei Zhang. Efficient structure-aware image smoothing by local extrema on space-filling curve. September 2014. p. 1254.
-